



London Live Bus Journey Tracker — Technical Assignment

1. Objectives & Core Challenge

Objective: Build a **native mobile application** for **Android (Jetpack Compose)** or **iOS (SwiftUI)** that enables a user to:

1. Plan a bus journey between two locations in London
2. Handle ambiguous locations and let the user disambiguate
3. Select a bus leg from a journey result
4. Track buses on that route in near-real time on a map

This assignment focuses on:

- Architecture & state management
- Data modelling & joining incomplete datasets
- Concurrency & lifecycle correctness
- Clear engineering reasoning and trade-offs

Visual polish or feature completeness is not a priority.

Core Engineering Challenge — “The Virtual GPS”

The TfL API **does not expose live latitude/longitude coordinates for buses**. Instead, your solution must infer vehicle position by **joining three independent datasets**:

1. **Journey Planning** – determines the relevant bus route
2. **Live Arrivals (Telemetry)** – indicates which stop a bus is approaching
3. **Route Sequence (Spatial Geometry)** – provides coordinates for stops along the route

This mirrors real-world systems where telemetry is incomplete and must be interpreted. The task is to derive bus positions on the map via the combined logic. Interpolation or animation is optional, reasonable approximation is acceptable.

2. Functional Requirements

2.1 Journey Planning & Location Disambiguation

Search API

GET /StopPoint/Search/{query}?modes=bus

Expected Behaviour

- Convert user-entered text into one or more potential StopPoints.
- If **one** clear match → auto-select.
- If **multiple** matches → show list → let the user choose.
- Either origin or destination may require disambiguation.

2.2 Journey Results

Journey Planning API

GET /Journey/JourneyResults/{from}/to/{to}

Expected Behaviour

- Detect:
 - Valid journey results, or
 - A disambiguation response
- Present disambiguation options if required
- Retry the request with chosen resolved IDs
- You only need to support **bus** legs
- User should select a **bus journey leg**

2.3 Live Telematics Dashboard

Once a bus route/leg is selected:

Arrivals API

GET /Line/{linId}/Arrivals

Required Handling

- Poll every ~30 seconds
- Show list of active buses
- Show empty state when no active vehicles
- Display:
 - `vehicleId`, `naptanId` & `timeToStation`

2.4 Map Visualisation

Route Geometry API

GET /Line/{lineId}/Route/Sequence/outbound

Requirements

- Match `naptanId` from telemetry to stop objects in route sequence
- Use stop coordinates to place buses on the map
- Use `timeToStation` to provide a sense of progress (simple hints ok)
- Smooth animation is **not required**

3. Technical Constraints & Expectations

3.1 Polling Lifecycle Management

- Refresh live data every 30 seconds
- Poll only when screen is visible
- Suspend polling when backgrounded

3.2 Concurrency

- Android → **Kotlin Coroutines + Flow**
- iOS → **Swift Concurrency (async/await)**
- UI must remain responsive during updates

3.3 Architecture & Data Management

- Use MVVM, MVI, or similar unidirectional state flow
- Keep concerns separated:
 - Network DTOs
 - Domain models
 - UI state
- Perform dataset joins in memory
- Do not re-fetch static geometry unnecessarily

3.4 Non-Goals

You **do not** need:

- Pixel-perfect UI
- Advanced map animation
- Offline support

- Authentication
- Persistent storage

The focus is correctness and clarity.

4. API Reference & Access Setup

4.1 Base API Endpoints (2026 Unified API)

Feature	Resource Path
Location Search	https://api.tfl.gov.uk/StopPoint/Search/{query}?modes=bus
Journey Planning	https://api.tfl.gov.uk/Journey/JourneyResults/{from}/to/{to}
Live Arrivals	https://api.tfl.gov.uk/Line/{lineId}/Arrivals
Route Geometry	https://api.tfl.gov.uk/Line/{lineId}/Route/Sequence/outbound

Append your key to all requests: [?app_key={{your_key}}](#)

4.2 API Key Setup

1. Create a TfL API account
<https://api-portal.tfl.gov.uk/>
2. Subscribe to a product
("500 Requests per Min" or "TrackernetFeedPublic")
3. Retrieve your **Primary Key** from your profile
→ This becomes your [app_key](#).
4. Use it in requests: → `app_key=YOUR_PRIMARY_KEY`. The old [app_id](#) is not required.

5. Screens (Reference Only)

5.1 Low-Fidelity Screens

These show:

- Journey planning flow
- Disambiguation screens
- Selection of bus route leg
- Live bus list
- Map with bus markers

Used to communicate structure and flow — **not** visual fidelity. See visuals at the end of the document.

5.2 High-Fidelity Screens (Optional Bonus)

These demonstrate:

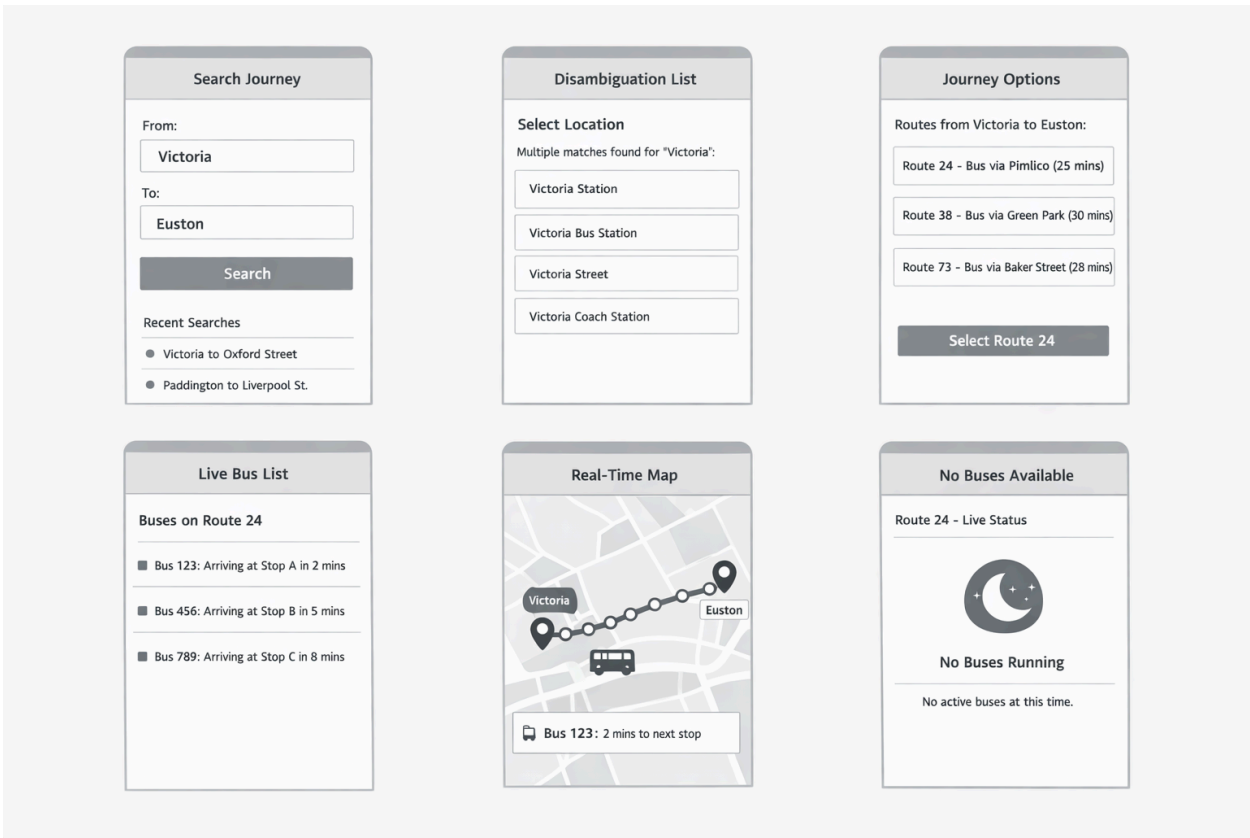
- Potential visual direction
- Component spacing
- Example use of a design system

Following Material 3 (Android) or iOS HIG is fully acceptable — matching these exactly is optional. See visuals at the end of the document.

Figma links (optional):

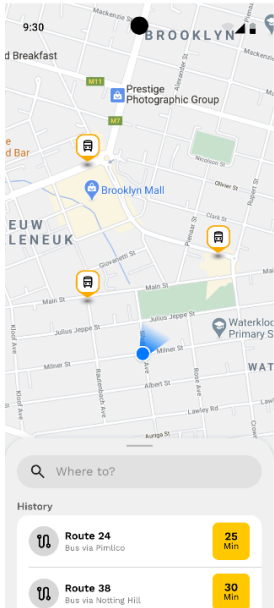
- [High-Fidelity App Designs](#) (request for access)
- [Design System](#) (optional bonus)

Low-Fidelity Screens

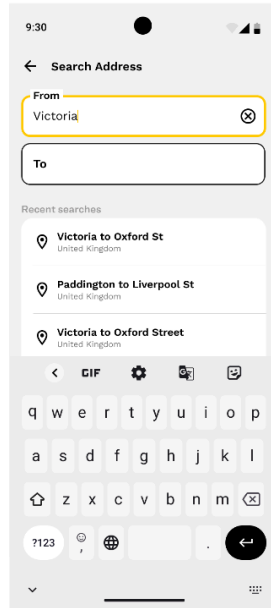


High-Fidelity Screens

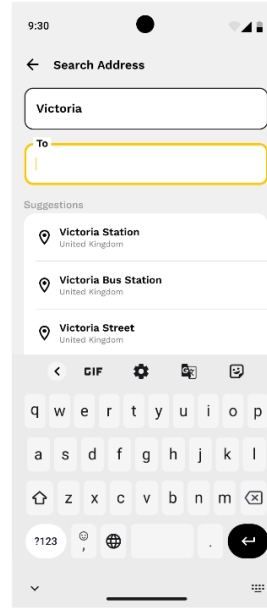
Designer
Landing



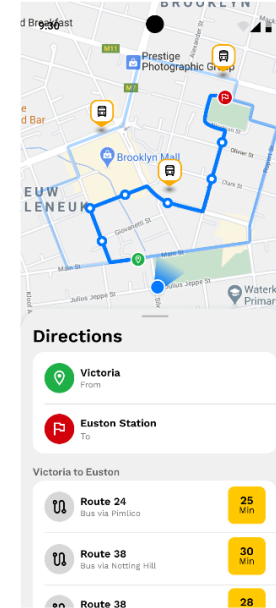
Designer
Search



Designer
Disambiguation list



Designer
Journey options



Designer

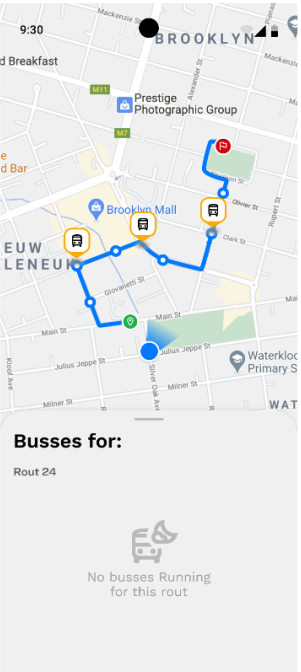
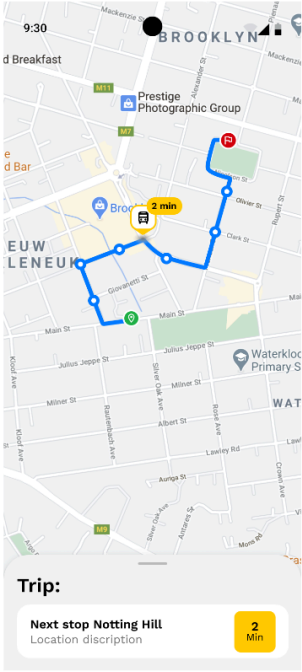
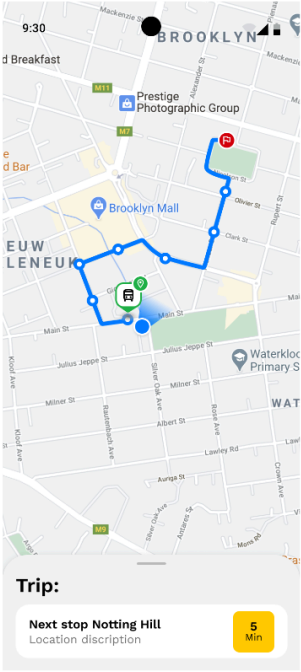
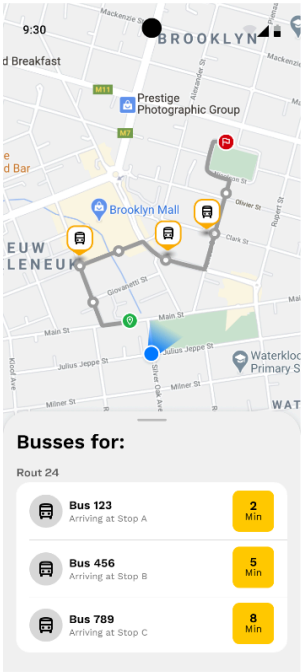
Bus list

Designer

Real time map

Designer

No busses available



UI component:

Bus states

Start



Bus are at the start location



Bus are on route and 2 min away

End



Bus are at the end location



Generic bus